

Title : Fraud Detection Using Machine Learning

Algorithm and Dataset — Technical Documentation

1. Algorithm

1.1 Model / Technique Applied

The fraud detection system uses a **hybrid supervised and unsupervised machine learning approach**. Supervised classification models learn from historically labeled fraud/non-fraud transactions, while unsupervised anomaly detection models flag unusual patterns that do not match known fraud signatures (useful for detecting new or evolving fraud types).

Category	Algorithm / Technique	Purpose
Supervised Learning	Logistic Regression	Baseline probability-based classification
Supervised Learning	Random Forest	Handles non-linear patterns, feature importance
Supervised Learning	XGBoost / Gradient Boosting	High-accuracy classification on imbalanced data
Unsupervised Learning	Isolation Forest	Detects outlier transactions without labels
Unsupervised Learning	Autoencoders (Neural Network)	Reconstructs normal patterns; high error = anomaly
Unsupervised Learning	DBSCAN / K-Means Clustering	Groups similar transactions, isolates outliers
Risk Scoring	Weighted Ensemble / Voting	Combines model outputs into a single fraud risk score

1.2 Step-by-Step Procedure

- Data Collection:** Gather transactional data from databases, APIs, or transaction logs (amount, timestamp, location, device, payment method, customer profile).
- Data Cleaning:** Handle missing values, remove duplicates, correct inconsistent formats, and treat outliers.
- Exploratory Data Analysis (EDA):** Analyze transaction distributions, fraud vs. non-fraud ratios, correlations, and time-based patterns using visualizations.
- Feature Engineering:** Create derived features such as transaction velocity, average spend per customer, distance between transaction locations, time since last transaction, and device/IP mismatch flags.
- Handling Class Imbalance:** Apply techniques such as SMOTE (Synthetic Minority Oversampling), undersampling, or class-weight adjustment, since fraud cases are typically a very small percentage of total transactions.
- Train-Test Split:** Divide the dataset into training and testing subsets (commonly 70:30 or 80:20) using stratified sampling to preserve fraud ratio.

- 7 **Model Training:** Train supervised models (Logistic Regression, Random Forest, XGBoost) on labeled data and unsupervised models (Isolation Forest, Autoencoders) on the full transaction set.
- 8 **Model Evaluation:** Assess performance using Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrix (accuracy alone is misleading on imbalanced data).
- 9 **Risk Scoring:** Combine model outputs into a normalized fraud risk score (0-100) using weighted ensembling or probability calibration.
- 10 **Alert Generation:** Transactions above a defined risk threshold are flagged for real-time alerts or manual investigation.
- 11 **Deployment & Monitoring:** Deploy the model into a real-time pipeline (streaming or batch), monitor performance drift, and retrain periodically with new labeled data.
- 12 **Feedback Loop:** Investigator decisions (confirmed fraud / false positive) are fed back into the training data to continuously improve model accuracy.

1.3 Logic / Method Used

The core logic combines **rule-based checks** (e.g., transaction amount exceeding a threshold, unusual geolocation, multiple rapid transactions) with **machine learning probability scoring**. Each transaction is passed through trained models that output a probability of being fraudulent. This probability is converted into a risk score and compared against a decision threshold: scores below the threshold are approved automatically, scores in a mid-range trigger step-up verification, and scores above the threshold are blocked or flagged for investigation.

1.4 Process Flow of the System

1	Transaction Data Input (real-time / batch)
2	Data Preprocessing & Feature Engineering
3	Rule-Based Filter (basic sanity checks)
4	ML Model Scoring (supervised + anomaly detection)
5	Risk Score Calculation & Aggregation
6	Threshold Decision (Approve / Review / Block)
7	Alert Generation & Dashboard Update
8	Investigator Feedback → Model Retraining Loop

2. Dataset

2.1 Source of Data

Data can be sourced from a combination of the following, depending on the deployment environment:

- Internal transaction databases (banking core systems, payment gateways, e-commerce order systems)
- Publicly available benchmark datasets for research/prototyping, e.g., the Kaggle **Credit Card Fraud Detection** dataset (European cardholder transactions) or the **IEEE-CIS Fraud Detection** dataset
- Third-party data providers (device fingerprinting, IP geolocation, credit bureau data)
- Customer relationship management (CRM) and KYC (Know Your Customer) systems

2.2 Number of Records / Samples

Dataset size varies by source and business scale; a typical prototype dataset contains **250,000 to 1,000,000+ transaction records**, with fraudulent cases typically representing only **0.1% to 2%** of total records (highly imbalanced). Production systems processing real-time streams can accumulate millions of records per month.

2.3 Features / Attributes (Columns)

Feature Category	Example Attributes
Transaction Details	Transaction ID, Amount, Currency, Timestamp, Merchant Category
Customer Profile	Customer ID, Account Age, Average Transaction Value, Credit Score
Payment Information	Payment Method, Card Type, Card Present/Not Present
Location Data	Billing Address, IP Address, Geolocation, Distance from Home
Device Information	Device ID, Device Type, Browser/OS, Device Fingerprint
Behavioral Features	Transaction Frequency, Time Since Last Transaction, Login Pattern
Historical Fraud Data	Previous Chargeback Flag, Prior Fraud Reports, Blacklist Status
Target Variable	Is_Fraud (0 = Legitimate, 1 = Fraudulent)

2.4 Training and Testing Data Split

Split Strategy	Ratio	Notes
Standard Split	80% Train / 20% Test	Common baseline split
Alternative Split	70% Train / 30% Test	Used with smaller datasets
Validation Split	10% of Train set	For hyperparameter tuning

Sampling Method	Stratified Sampling	Preserves fraud/non-fraud ratio across splits
Cross-Validation	K-Fold (k=5 or 10)	Ensures robust performance estimation

2.5 Data Format / Type

- **File Format:** CSV, JSON, or Parquet for batch data; JSON/Avro for streaming (Kafka) sources
- **Storage:** Relational (SQL: MySQL, PostgreSQL) for structured transaction records; NoSQL (MongoDB, Cassandra) for device/behavioral logs; Data lakes (Hadoop/Spark) for large-scale historical data
- **Data Types:** Numerical (amount, account age), Categorical (payment method, merchant category), Datetime (timestamp), Boolean (card present flag), Text/ID (transaction ID, device ID)

Note: Figures and dataset sources above are representative examples used commonly in fraud analytics projects. Replace with actual organizational data source, record counts, and split ratios once the specific dataset is finalized.

3. Source Code

The complete Python implementation of the fraud detection pipeline is provided below, covering data loading, cleaning, exploratory data analysis, feature engineering, model training (supervised and unsupervised), evaluation, risk scoring, and alert generation. Each function is documented with its purpose, inputs, and outputs.

```
"""
=====
FRAUD DETECTION ANALYTICS SYSTEM
Complete Source Code
=====
Description : End-to-end fraud detection pipeline covering data
              loading, cleaning, EDA, feature engineering, model
              training (supervised + unsupervised), evaluation,
              risk scoring, and real-time-style alert generation.
Language    : Python 3
Libraries   : pandas, numpy, scikit-learn, matplotlib, seaborn
=====
"""

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg") # non-interactive backend (safe for scripts/servers)
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix, classification_report
)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# -----
# 1. DATA LOADING
# -----
def load_data(filepath: str = None) -> pd.DataFrame:
    """
    Load transaction data from a CSV file.
    If no filepath is provided, a synthetic sample dataset is
    generated so the pipeline can run end-to-end for demo/testing.

    Parameters
    -----
    filepath : str, optional
        Path to the transactions CSV file.

    Returns
    -----
    pd.DataFrame
        Raw transaction data.
    """
    if filepath:
        df = pd.read_csv(filepath)
        return df

    # ---- Synthetic dataset generator (for demo purposes only) ----
    n = 5000
    fraud_ratio = 0.02
    n_fraud = int(n * fraud_ratio)
    n_normal = n - n_fraud

    def generate_records(count, is_fraud):
        return pd.DataFrame({
```

```

        "transaction_id": np.arange(count),
        "amount": np.round(
            np.random.exponential(scale=250 if is_fraud else 80, size=count), 2
        ),
        "hour_of_day": np.random.randint(0, 24, size=count),
        "account_age_days": np.random.randint(1, 3000, size=count),
        "distance_from_home_km": np.round(
            np.random.exponential(scale=180 if is_fraud else 15, size=count), 2
        ),
        "transactions_last_24h": np.random.poisson(
            lam=6 if is_fraud else 1.5, size=count
        ),
        "payment_method": np.random.choice(
            ["credit_card", "debit_card", "net_banking", "wallet"], size=count
        ),
        "device_type": np.random.choice(
            ["mobile", "desktop", "pos_terminal"], size=count
        ),
        "is_fraud": is_fraud
    })

df = pd.concat([
    generate_records(n_normal, 0),
    generate_records(n_fraud, 1)
], ignore_index=True)

df = df.sample(frac=1, random_state=RANDOM_STATE).reset_index(drop=True)
return df

```

```

# -----
# 2. DATA CLEANING
# -----
def clean_data(df: pd.DataFrame) -> pd.DataFrame:
    """
    Clean raw transaction data:
    - Drop duplicate transactions
    - Handle missing values
    - Remove invalid/negative amounts

    Parameters
    -----
    df : pd.DataFrame
        Raw transaction data.

    Returns
    -----
    pd.DataFrame
        Cleaned transaction data.
    """
    df = df.drop_duplicates(subset="transaction_id").copy()

    # Fill missing numeric values with median, categorical with mode
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    categorical_cols = df.select_dtypes(include=["object"]).columns

    for col in numeric_cols:
        df[col] = df[col].fillna(df[col].median())
    for col in categorical_cols:
        df[col] = df[col].fillna(df[col].mode()[0])

    # Remove invalid transactions
    df = df[df["amount"] > 0].reset_index(drop=True)

    return df

```

```

# -----
# 3. EXPLORATORY DATA ANALYSIS (EDA)
# -----
def run_eda(df: pd.DataFrame, save_path: str = None) -> None:
    """
    Generate summary statistics and visualizations to understand
    transaction patterns and fraud distribution.

    Parameters

```

```

-----
df : pd.DataFrame
    Cleaned transaction data.
save_path : str, optional
    If provided, saves the EDA figure to this path instead of
    displaying it (useful for headless/server environments).
"""
print("Dataset shape:", df.shape)
print("\nFraud distribution:\n", df["is_fraud"].value_counts(normalize=True))
print("\nSummary statistics:\n", df.describe())

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

sns.countplot(x="is_fraud", data=df, ax=axes[0, 0])
axes[0, 0].set_title("Fraud vs Legitimate Transaction Count")

sns.boxplot(x="is_fraud", y="amount", data=df, ax=axes[0, 1])
axes[0, 1].set_title("Transaction Amount by Fraud Label")

sns.histplot(data=df, x="hour_of_day", hue="is_fraud", bins=24,
             multiple="stack", ax=axes[1, 0])
axes[1, 0].set_title("Transaction Hour Distribution")

corr = df.select_dtypes(include=[np.number]).corr()
sns.heatmap(corr, annot=False, cmap="coolwarm", ax=axes[1, 1])
axes[1, 1].set_title("Feature Correlation Heatmap")

plt.tight_layout()
if save_path:
    plt.savefig(save_path)
    plt.close()
else:
    plt.show()

# -----
# 4. FEATURE ENGINEERING
# -----
def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    """
    Create derived features that improve fraud signal strength.

    Parameters
    -----
    df : pd.DataFrame
        Cleaned transaction data.

    Returns
    -----
    pd.DataFrame
        DataFrame with additional engineered features.
    """
    df = df.copy()

    # Flag unusually large transactions (relative to overall mean)
    df["high_amount_flag"] = (df["amount"] > df["amount"].mean() +
                             2 * df["amount"].std()).astype(int)

    # Flag transactions made late at night (common fraud window)
    df["night_transaction_flag"] = df["hour_of_day"].apply(
        lambda h: 1 if (h < 5 or h > 22) else 0
    )

    # Flag high-velocity accounts (many transactions in 24h)
    df["high_velocity_flag"] = (df["transactions_last_24h"] > 5).astype(int)

    # Flag new accounts (higher fraud risk)
    df["new_account_flag"] = (df["account_age_days"] < 30).astype(int)

    # Encode categorical variables
    for col in ["payment_method", "device_type"]:
        le = LabelEncoder()
        df[col + "_enc"] = le.fit_transform(df[col])

    return df

```

```

# -----
# 5. TRAIN / TEST SPLIT
# -----
def split_data(df: pd.DataFrame, feature_cols: list, target_col: str = "is_fraud"):
    """
    Split the dataset into training and testing sets using
    stratified sampling to preserve the fraud ratio, then scale
    numeric features.

    Parameters
    -----
    df : pd.DataFrame
        Feature-engineered dataset.
    feature_cols : list
        List of feature column names to use for modeling.
    target_col : str
        Name of the target/label column.

    Returns
    -----
    tuple
        X_train, X_test, y_train, y_test, scaler
    """
    X = df[feature_cols]
    y = df[target_col]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    return X_train_scaled, X_test_scaled, y_train, y_test, scaler

# -----
# 6. MODEL TRAINING (SUPERVISED)
# -----
def train_logistic_regression(X_train, y_train) -> LogisticRegression:
    """Train a Logistic Regression baseline classifier."""
    model = LogisticRegression(
        class_weight="balanced", max_iter=1000, random_state=RANDOM_STATE
    )
    model.fit(X_train, y_train)
    return model

def train_random_forest(X_train, y_train) -> RandomForestClassifier:
    """Train a Random Forest classifier."""
    model = RandomForestClassifier(
        n_estimators=200,
        max_depth=10,
        class_weight="balanced",
        random_state=RANDOM_STATE,
        n_jobs=-1
    )
    model.fit(X_train, y_train)
    return model

# -----
# 7. MODEL TRAINING (UNSUPERVISED ANOMALY DETECTION)
# -----
def train_isolation_forest(X_train, contamination: float = 0.02) -> IsolationForest:
    """
    Train an Isolation Forest to detect anomalous transactions
    without relying on fraud labels.

    Parameters
    -----
    X_train : array-like
        Training feature matrix.
    contamination : float

```

Expected proportion of anomalies in the data.

Returns

IsolationForest

Trained anomaly detection model.

"""

```
model = IsolationForest(
    n_estimators=200,
    contamination=contamination,
    random_state=RANDOM_STATE
)
model.fit(X_train)
return model
```

8. MODEL EVALUATION

def evaluate_model(model, X_test, y_test, model_name: str = "Model") -> dict:

"""

Evaluate a trained classifier using standard fraud-detection metrics (precision, recall, F1, ROC-AUC are prioritized over plain accuracy because of class imbalance).

Parameters

model : sklearn estimator
Trained classification model.
X_test, y_test : array-like
Test features and true labels.
model_name : str
Label used when printing the report.

Returns

dict

Dictionary of evaluation metrics.

"""

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1] if hasattr(model, "predict_proba") else y_pred
```

```
metrics = {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred, zero_division=0),
    "recall": recall_score(y_test, y_pred, zero_division=0),
    "f1_score": f1_score(y_test, y_pred, zero_division=0),
    "roc_auc": roc_auc_score(y_test, y_prob)
}
```

```
print(f"\n--- {model_name} Evaluation ---")
for k, v in metrics.items():
    print(f"{k:>10}: {v:.4f}")
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))
```

return metrics

9. RISK SCORING (ENSEMBLE)

def compute_risk_score(rf_model, iso_model, X) -> np.ndarray:

"""

Combine the Random Forest fraud probability and the Isolation Forest anomaly score into a single 0-100 risk score per transaction.

Parameters

rf_model : RandomForestClassifier
Trained supervised model.
iso_model : IsolationForest
Trained anomaly detection model.
X : array-like
Feature matrix to score.

```

Returns
-----
np.ndarray
    Array of risk scores (0-100) for each transaction.
"""
rf_prob = rf_model.predict_proba(X)[: , 1]

# Isolation Forest: more negative score_samples => more anomalous.
# Normalize to a 0-1 "anomaly probability".
raw_scores = iso_model.score_samples(X)
anomaly_prob = (raw_scores.max() - raw_scores) / (raw_scores.max() - raw_scores.min())

# Weighted ensemble: 70% supervised model, 30% anomaly detector
combined = 0.7 * rf_prob + 0.3 * anomaly_prob
risk_score = np.round(combined * 100, 2)
return risk_score

# -----
# 10. ALERT GENERATION
# -----
def generate_alerts(df: pd.DataFrame, risk_scores: np.ndarray,
                   high_threshold: float = 75, review_threshold: float = 40) -> pd.DataFrame:
    """
    Classify transactions into decision buckets based on risk score
    and attach the results to the original transaction records.

    Parameters
    -----
    df : pd.DataFrame
        Original transaction records (aligned with risk_scores).
    risk_scores : np.ndarray
        Computed risk scores for each transaction.
    high_threshold : float
        Score at or above which a transaction is blocked/flagged.
    review_threshold : float
        Score at or above which a transaction requires manual review.

    Returns
    -----
    pd.DataFrame
        Transactions with 'risk_score' and 'decision' columns.
    """
    result = df.copy()
    result["risk_score"] = risk_scores

    def decide(score):
        if score >= high_threshold:
            return "BLOCK / INVESTIGATE"
        elif score >= review_threshold:
            return "MANUAL REVIEW"
        else:
            return "APPROVE"

    result["decision"] = result["risk_score"].apply(decide)
    return result

# -----
# 11. MAIN PIPELINE
# -----
def main():
    """Run the complete fraud detection pipeline end-to-end."""

    # Step 1: Load data
    df_raw = load_data()

    # Step 2: Clean data
    df_clean = clean_data(df_raw)

    # Step 3: EDA (saved to file instead of shown interactively)
    run_eda(df_clean, save_path="eda_report.png")

    # Step 4: Feature engineering
    df_features = engineer_features(df_clean)

```

```

feature_cols = [
    "amount", "hour_of_day", "account_age_days",
    "distance_from_home_km", "transactions_last_24h",
    "high_amount_flag", "night_transaction_flag",
    "high_velocity_flag", "new_account_flag",
    "payment_method_enc", "device_type_enc"
]

# Step 5: Train/test split
X_train, X_test, y_train, y_test, scaler = split_data(df_features, feature_cols)

# Step 6: Train supervised models
lr_model = train_logistic_regression(X_train, y_train)
rf_model = train_random_forest(X_train, y_train)

# Step 7: Train unsupervised anomaly model
iso_model = train_isolation_forest(X_train)

# Step 8: Evaluate models
evaluate_model(lr_model, X_test, y_test, "Logistic Regression")
evaluate_model(rf_model, X_test, y_test, "Random Forest")

# Step 9: Compute risk scores on the test set
risk_scores = compute_risk_score(rf_model, iso_model, X_test)

# Step 10: Generate alerts / decisions
test_records = df_features.iloc[y_test.index].reset_index(drop=True)
alert_report = generate_alerts(test_records, risk_scores)

print("\nSample Alert Report:\n", alert_report[
    ["transaction_id", "amount", "risk_score", "decision"]
].head(10))

alert_report.to_csv("fraud_alert_report.csv", index=False)
print("\nPipeline complete. Alert report saved to 'fraud_alert_report.csv'.")

if __name__ == "__main__":
    main()

```

The full source code is also provided as a standalone, runnable file: **`fraud_detection_source.py`**.